

Worksheet

Computer Network - Week 10 - Memory Management

Name	Farrel Tsaqif Anindyo
Class	CSA

Activity 1

Procedure

Read the procedure on the activity file.

```
(farrelta@DESKTOP-9TBCDM7) - [~/Downloads]
$ ./stack_heap.out
=== Stack Allocation ===
Stack array address: 0x7ffe06e42930
Stack array address (end): 0x7ffe06e438cc
Stack array size: 4000 bytes

=== Heap Allocation ===
Heap array address: 0x56fb1b7386b0
Heap array address (end): 0x56fb1b73964c
Heap array size: 4000 bytes

Process ID: 886
Sleeping for 30 seconds to allow inspection...
```

```

farrelta@DESKTOP-9TBCDM7: ~
$ cat /proc/886/maps
56fb09ae0000-56fb09ae1000 r--p 00000000 08:30 399810 /home/farrelta/Downloads/stack_heap.out
56fb09ae1000-56fb09ae2000 r-xp 00001000 08:30 399810 /home/farrelta/Downloads/stack_heap.out
56fb09ae2000-56fb09ae3000 r--p 00002000 08:30 399810 /home/farrelta/Downloads/stack_heap.out
56fb09ae3000-56fb09ae4000 r--p 00002000 08:30 399810 /home/farrelta/Downloads/stack_heap.out
56fb09ae4000-56fb09ae5000 rw-p 00003000 08:30 399810 /home/farrelta/Downloads/stack_heap.out
56fb1b738000-56fb1b759000 rw-p 00000000 00:00 0 [heap]
7d7ff3b21000-7d7ff3b24000 rw-p 00000000 00:00 0
7d7ff3b24000-7d7ff3b4c000 r--p 00000000 08:30 32796 /usr/lib/x86_64-linux-gnu/libc.so.6
7d7ff3b4c000-7d7ff3cb1000 r-xp 00002000 08:30 32796 /usr/lib/x86_64-linux-gnu/libc.so.6
7d7ff3cb1000-7d7ff3d07000 r--p 0018d000 08:30 32796 /usr/lib/x86_64-linux-gnu/libc.so.6
7d7ff3d07000-7d7ff3d0b000 r--p 001e2000 08:30 32796 /usr/lib/x86_64-linux-gnu/libc.so.6
7d7ff3d0b000-7d7ff3d0d000 rw-p 001e6000 08:30 32796 /usr/lib/x86_64-linux-gnu/libc.so.6
7d7ff3d0d000-7d7ff3d1a000 rw-p 00000000 00:00 0
7d7ff3d1a000-7d7ff3d33000 rw-p 00000000 00:00 0
7d7ff3d33000-7d7ff3d34000 r--p 00000000 08:30 32732 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d7ff3d34000-7d7ff3d5c000 r-xp 00001000 08:30 32732 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d7ff3d5c000-7d7ff3d67000 r--p 00029000 08:30 32732 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d7ff3d67000-7d7ff3d69000 r--p 00034000 08:30 32732 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d7ff3d69000-7d7ff3d6a000 rw-p 00036000 08:30 32732 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
7d7ff3d6a000-7d7ff3d6b000 rw-p 00000000 00:00 0
7ffe06e24000-7ffe06e45000 rw-p 00000000 00:00 0 [stack]
7ffe06f25000-7ffe06f29000 r--p 00000000 00:00 0 [vvar]
7ffe06f29000-7ffe06f2b000 r-xp 00000000 00:00 0 [vdso]

```

Answers

1. Before running the program, predict: will the stack array address be numerically larger or smaller than the heap array address? Explain your reasoning based on the virtual address space layout you learned in the module.

I think the stack will be larger than the heap because the stack starts at the "top" and the Heap starts near the "bottom," the stack will have significantly larger hexadecimal values compared to heap. Also the size of the heap is dynamic, so it could be bigger or lower.

2. Run the program and open `/proc/<PID>/maps`. Were you correct? Identify which region corresponds to the stack and which to the heap. Look at the permissions (e.g., `rw-p`, `r-xp`): why doesn't the stack region have execute permission? What would typically happen if a program tried to execute code stored on the stack?

Yes, i was correct,
Stack hex: 7ffe06e24000
Heap hex: 56fb1b738000
The "x" is present in heap because the CPU must be allowed to execute the compiled code,
For protection, if an attacker tries to inject code into your stack_array and jump to it, the CPU will trigger a Segmentation Fault immediately.

Activity 2

Procedure

Read the procedure on the activity file.

```
farrelta@DESKTOP-9TBCEM7 x  farrelta@DESKTOP-9TBCEM7: x  + v
(farrelta@DESKTOP-9TBCEM7) ~[~/Downloads]
$ ./memory_leak.out
=== Memory Leak Experiment ===
Process ID (PID): 1056
This program will allocate memory repeatedly and NOT free it.
Use 'top -p <PID>' in another terminal to observe VIRT and RES.

Iteration 1: allocated 1048576 bytes (total leaked so far: ~1 MB)
Iteration 2: allocated 1048576 bytes (total leaked so far: ~2 MB)
Iteration 3: allocated 1048576 bytes (total leaked so far: ~3 MB)
Iteration 4: allocated 1048576 bytes (total leaked so far: ~4 MB)
Iteration 5: allocated 1048576 bytes (total leaked so far: ~5 MB)
Iteration 6: allocated 1048576 bytes (total leaked so far: ~6 MB)
Iteration 7: allocated 1048576 bytes (total leaked so far: ~7 MB)
Iteration 8: allocated 1048576 bytes (total leaked so far: ~8 MB)
Iteration 9: allocated 1048576 bytes (total leaked so far: ~9 MB)
Iteration 10: allocated 1048576 bytes (total leaked so far: ~10 MB)
Iteration 11: allocated 1048576 bytes (total leaked so far: ~11 MB)
Iteration 12: allocated 1048576 bytes (total leaked so far: ~12 MB)
Iteration 13: allocated 1048576 bytes (total leaked so far: ~13 MB)
Iteration 14: allocated 1048576 bytes (total leaked so far: ~14 MB)
Iteration 15: allocated 1048576 bytes (total leaked so far: ~15 MB)
Iteration 16: allocated 1048576 bytes (total leaked so far: ~16 MB)
Iteration 17: allocated 1048576 bytes (total leaked so far: ~17 MB)
Iteration 18: allocated 1048576 bytes (total leaked so far: ~18 MB)
Iteration 19: allocated 1048576 bytes (total leaked so far: ~19 MB)
Iteration 20: allocated 1048576 bytes (total leaked so far: ~20 MB)

Done allocating. Sleeping for 60 seconds so you can still observe in 'top'...
```

```
farrelta@DESKTOP-9TBCDM7: x  farrelta@DESKTOP-9TBCDM7 x  +  v
top - 14:06:17 up 29 min, 1 user, load average: 0.01, 0.02, 0.00
Tasks: 1 total, 0 running, 1 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.2 us, 0.4 sy, 0.0 ni, 99.4 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1844.2 total, 1275.6 free, 430.6 used, 214.1 buff/cache
MiB Swap: 1024.0 total, 1024.0 free, 0.0 used, 1413.7 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 1056 farrelta  20   0 23120 22144 1536 S   0.0   1.2   0:00.22 memory_leak.out
```

Answers

1. Before running the program, predict: what will happen to the **VIRT** (virtual memory) and **RES** (resident memory) values in top as the program runs? Will they increase, decrease, or stay the same? Why?

I think both VIRT (Virtual Memory) and RES (Resident Memory) will increase by approximately 1 MB (1024 KB) every second because when you call "malloc(1MB)", the program asks the OS for address space and when you call "memset", you are actually writing data to that memory.

2. Observe the top output while the program runs. Describe the pattern in **VIRT** and **RES**. Do they both grow at the same rate? If not, explain the difference. What does this tell you about how the OS manages virtual vs physical memory?

Yes, they grow at the same time

It tells me that,

- a. Virtual vs. Physical Separation: The OS maintains a mapping table (Page Table) for the process. malloc only affects the Virtual side of this map. It is essentially a "promise" of memory.
- b. The "Lazy" OS: The OS is optimistic. It assumes you might ask for 1 MB but only use 10 KB. To save RAM, it does not give you physical frames when you malloc. It only gives you the addresses.
- c. Page Faults: When your code actually tries to write to that address (the memset), the CPU triggers a hardware exception called a Page Fault. The OS catches this, sees that you have

permission to use that memory, pauses your program, finds a free page of physical RAM, maps it to your virtual address, and then lets your program resume.

Activity 3

Procedure

Read the procedure on the activity file.

```
(farrelta@DESKTOP-9TBCDM7) - [~/Downloads]
$ ./dangerous_memory.out
=== Scenario 1: Use-After-Free ===
Before free: data = Important data (address: 0x5f7c983656b0)
After free: attempting to access data...
Data after free: e♦♦♦
Modified data: Modified after free

=== Scenario 2: Double Free ===
Buffer: Buffer content
First free completed
free(): double free detected in tcache 2
Aborted
```

Answers

1. Run the program and observe the output. In Scenario 1, does the program always crash after the **free**? If it still prints "Important data", explain how this can happen even though the memory has been freed. Why is this behavior dangerous in real applications?

Yes, the data has been modified,
Because when you call free(data), the allocator doesn't just mark the memory as "free" in a spreadsheet. It needs to store this free block in a Linked List so it can find it later.
If the program continued running and tried to allocate more memory, it would likely crash because you overwrote the "Next Pointer" with your text.

2. In Scenario 2, does the double free cause a crash or error message on your system? If yes, what is the message? If no, why might it still be a serious bug? Briefly explain what a "double free vulnerability" is.

No, because it was automatically aborted in my system

Because it is to prevent you from corrupting the heap structure (which hackers use to take over systems)

Double free errors occur when `free()` is called more than once with the same memory address as an argument. Calling `free()` twice on the same value can lead to memory leak

Activity 4

Procedure

Read the procedure on the activity file.

```
farrelta@DESKTOP-9TB8CDM7: X  farrelta@DESKTOP-9TB8CDM7: X  +  v
[farrelta@DESKTOP-9TB8CDM7] ~/Downloads
$ ./buffer_test.out
=== Buffer Overflow Experiment ===

Allocated buffer for 5 integers
Buffer address: 0x56d4871046b0
Buffer size: 20 bytes

Writing to valid indices (0-4):
buffer[0] = 0 (address: 0x56d4871046b0)
buffer[1] = 10 (address: 0x56d4871046b4)
buffer[2] = 20 (address: 0x56d4871046b8)
buffer[3] = 30 (address: 0x56d4871046bc)
buffer[4] = 40 (address: 0x56d4871046c0)

Now writing beyond the allocated buffer...
(This is dangerous and should never be done!)

Attempting to write to buffer[5] (address: 0x56d4871046c4)...
Write succeeded! Value: 50
Attempting to write to buffer[6] (address: 0x56d4871046c8)...
Write succeeded! Value: 60
Attempting to write to buffer[7] (address: 0x56d4871046cc)...
Write succeeded! Value: 70
Attempting to write to buffer[8] (address: 0x56d4871046d0)...
Write succeeded! Value: 80
Attempting to write to buffer[9] (address: 0x56d4871046d4)...
Write succeeded! Value: 90

Reading all values (including out-of-bounds):
buffer[0] = 0
buffer[1] = 10
buffer[2] = 20
buffer[3] = 30
buffer[4] = 40
```

```
buffer[5] = 50
buffer[6] = 60
buffer[7] = 70
buffer[8] = 80
buffer[9] = 90
```

Program completed. Did it crash?

```
farrelta@DESKTOP-9TB8CDM7 X farrelta@DESKTOP-9TB8CDM7: X + v
=== Buffer Overflow Experiment ===

Allocated buffer for 5 integers
Buffer address: 0x6e7b0e2e0040
Buffer size: 20 bytes

Writing to valid indices (0-4):
buffer[0] = 0 (address: 0x6e7b0e2e0040)
buffer[1] = 10 (address: 0x6e7b0e2e0044)
buffer[2] = 20 (address: 0x6e7b0e2e0048)
buffer[3] = 30 (address: 0x6e7b0e2e004c)
buffer[4] = 40 (address: 0x6e7b0e2e0050)

Now writing beyond the allocated buffer...
(This is dangerous and should never be done!)

Attempting to write to buffer[5] (address: 0x6e7b0e2e0054)...
=====
==1342==ERROR: AddressSanitizer: heap-buffer-overflow on address 0x6e7b0e2e0054 at pc 0x59ea64bcb3e9 bp 0x7ffd0e903760 sp 0x7ffd0e903758
WRITE of size 4 at 0x6e7b0e2e0054 thread T0
#0 0x59ea64bcb3e8 in main (/home/farrelta/Downloads/buffer_test.out+0x13e8) (BuildId: 4264d729ed2a6bbfbbf7f7adbee89eaa387f35f)
#1 0x724b0f033ca7 (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId: def5460e3cee00bfee25b429c97bcc4853e5b3a8)
#2 0x724b0f033d64 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x29d64) (BuildId: def5460e3cee00bfee25b429c97bcc4853e5b3a8)
#3 0x59ea64bcb100 in _start (/home/farrelta/Downloads/buffer_test.out+0x1100) (BuildId: 4264d729ed2a6bbfbbf7f7adbee89eaa387f35f)

0x6e7b0e2e0054 is located 0 bytes after 20-byte region [0x6e7b0e2e0040,0x6e7b0e2e0054)
allocated by thread T0 here:
#0 0x724b0f31a0ab in malloc ../../src/libsanitizer/asan/asan_malloc_linux.cpp:67
#1 0x59ea64bcb1e9 in main (/home/farrelta/Downloads/buffer_test.out+0x11e9) (BuildId: 4264d729ed2a6bbfbbf7f7adbee89eaa387f35f)
#2 0x724b0f033ca7 (/lib/x86_64-linux-gnu/libc.so.6+0x29ca7) (BuildId: def5460e3cee00bfee25b429c97bcc4853e5b3a8)

SUMMARY: AddressSanitizer: heap-buffer-overflow (/home/farrelta/Downloads/buffer_test.out+0x13e8) (BuildId: 4264d729ed2a6bbfbbf7f7adbee89eaa387f35f)
) in main
Shadow bytes around the buggy address:
0x6e7b0e2dfdb0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6e7b0e2dfdb8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6e7b0e2dfdc0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x6e7b0e2dfdc8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
=>0x6e7b0e2e0000: fa fa 00 00 00 fa fa fa 00 00[04]fa fa fa fa fa
0x6e7b0e2e0008: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6e7b0e2e0010: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6e7b0e2e0018: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6e7b0e2e0020: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6e7b0e2e0028: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
0x6e7b0e2e0030: fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa fa
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==1342==ABORTING

(farrelta@DESKTOP-9TB8CDM7) ~/Downloads
$
```

Answers

1. Before running the program, predict: When the program tries to write to buffer[5] through buffer[9], will it crash immediately? Why or why not? Consider what you learned about memory allocation and page boundaries.

No, the program will likely not crash immediately while writing to `buffer[5]` through `buffer[9]`.

1. Page Granularity: The OS manages memory in 4096-byte pages. the 20-byte buffer and the few bytes will almost certainly reside within the same valid, writable page. Since it does not cross a page boundary into unmapped memory, the hardware (MMU) sees no error.
2. Heap Metadata: it silently overwrites the chunk metadata (hidden headers) that `malloc` uses to track the next block of memory.

2. Recompile the program with the following command to enable address sanitizer: `gcc -fsanitize=address buffer_test.c -o buffer_test.out`. Run it again and take a screenshot. What additional information does the address sanitizer provide? Why are tools like this essential for detecting memory bugs that might not cause immediate crashes?

It explicitly identified a heap-buffer-overflow, distinguishing it from generic segmentation faults. Without address sanitizer, the program silently corrupted memory and kept running. address sanitizer forces a crash the exact moment the illegal write occurs.